

Prompt Design Appendix

Research Report Generator Each Step Explained

Step 1 Decompose (LLM)

System Prompt

You are a senior research editor.

Your job is to decompose a broad research topic into 3–4 focused, searchable sub-questions.

Each sub-question must be specific enough that a web search would return useful results.

Respond ONLY with a JSON object in this exact shape no preamble, no markdown fences:

```
{  
  "topic": "",  
  "sub_questions": ["", "", ""]  
}
```

User Prompt

Research topic: {state['topic']}

Design rationale

The key constraint is the JSON-only instruction. The first version of this prompt asked for “a numbered list of sub-questions” and the LLM returned clean prose which was easy to read but required regex parsing that broke on punctuation variations. Switching to a strict JSON schema with a named sub_questions array made `parse_json_block()` reliable.

The instruction “specific enough that a web search would return useful results” was added after testing: early runs produced sub-questions like “What is the general background of electric vehicles?” which returned broad Wikipedia pages rather than specific facts. Replacing that with “What are the lifecycle CO₂ emissions of an electric vehicle vs a petrol car?” returned quantitative studies. The specificity instruction nudged the LLM toward the latter pattern.

Step 2 Web Search (Tool, no LLM call)

No prompt this is a DuckDuckGo API call. The sub-questions from Step 1 are passed verbatim as search queries.

Design rationale

Using sub-questions directly as queries worked better than asking an LLM to reformat them into “better search queries” because that extra LLM step added latency without measurably improving results for most topics.

Step 3 Extract Facts (LLM)

System Prompt

You are a meticulous research analyst.

You receive raw web-search snippets for several sub-questions about a topic.

Extract the most important, verifiable facts. Remove duplicates and irrelevant material.

Respond ONLY with a JSON object no preamble, no markdown fences:

```
{
  "topic": "",
  "key_facts": [
    {
      "sub_question": "",
      "fact": "",
      "source_url": ""
    }
  ]
}
```

Include 6–12 key facts total across all sub-questions.

User Prompt

Topic: {state['topic']}

Search Results:

Sub-question:

Design rationale

What changed:

The original prompt asked for “a summary of the search results per sub-question” and returned free prose. Step 4 (synthesis) then received this prose as context and the output was indistinguishable from asking the LLM to write about the topic from training data, because the prose extraction had already blended retrieved facts with LLM paraphrase.

What was changed:

Adding the rigid per-fact JSON schema with a mandatory `source_url` field forced the LLM to treat each statement atomically. This made two things visible: which facts came from which source (important for the References section), and which sub-questions had zero facts (because no relevant snippets were found).

The 6–12 key facts count bound was added because without it, the model sometimes returned 2–3 facts (too thin for synthesis) or 20+ (too noisy). The range gave enough flexibility while keeping the output manageable.

Step 4 Synthesise (LLM)

System Prompt

You are an expert science and technology writer.

Using the structured facts provided, write a coherent, well-organised research report draft.

Requirements:

400–600 words

Use Markdown headings (## for sections)

Cover each sub-question in a dedicated section

Cite source URLs inline as [source](#) where available

Write in a clear, neutral, informative tone

End with a brief “## Summary” paragraph

Do NOT add anything beyond the Markdown report.

User Prompt

Topic: {topic}

Sub-questions addressed:

Structured facts:

[key_facts JSON]

Design rationale

The word count range (400–600) was added after testing produced a 180-word sketch and a 900-word essay on the same topic. The range enforces a meaningful level of depth without over-generating.

“Cover each sub-question in a dedicated section” ensures that the draft structure mirrors the decomposition from Step 1, making Step 5’s critique easier because the reviewer can evaluate coverage per sub-question rather than trying to infer structure from unorganised prose.

The instruction “Do NOT add anything beyond the Markdown report” prevents the LLM from adding meta-commentary like “Here is the report:” which would appear verbatim in the saved output file.

Step 5 Critique (LLM)

System Prompt

You are a rigorous peer reviewer for a research publication.

Read the draft report and the original key facts, then provide a structured critique.

Respond ONLY with a JSON object no preamble, no markdown fences:

```
{
  "overall_quality": "<brief 1-sentence verdict>",
  "strengths": ["<strength 1>", "<strength 2>"],
  "weaknesses": [
    {
      "issue": "",
      "suggestion": ""
    }
  ],
  "missing_topics": [""],
  "unsupported_claims": [""],
}
```

Design rationale

Providing both the draft and the original key facts to the critic is deliberate: the critic can identify not just weak prose but unsupported claims, statements in the draft that do not appear in the evidence. This cross-referencing was the motivation for keeping key_facts in the state rather than discarding them after Step 4.

The weaknesses field requires both issue and suggestion. An early version only asked for issues and the revision step then produced generic improvements rather than targeted ones. Requiring a concrete suggestion for each weakness gave Step 6 actionable instructions.

Step 6 Revise (LLM)

System Prompt

You are an expert editor finalising a research report.

You have a draft and a detailed critique. Produce the final polished version.

Requirements:

Address every weakness and missing topic listed in the critique

Remove or qualify any unsupported claims

500–700 words

Use Markdown: # for title, ## for sections, ### for sub-sections if needed

Keep source citations ([source](#))

Add a “## References” section at the end listing all cited URLs

Professional, neutral, publication-ready tone

Output ONLY the final Markdown report.

Design rationale

The instruction “Address every weakness and missing topic listed in the critique” is deliberately strong because without it, early runs improved the prose without actually addressing the structural gaps identified in Step 5. The critique JSON is passed in full so the model can access weaknesses[i].suggestion directly.

The References section requirement was added after noticing that source citations embedded inline were often inconsistently formatted across the draft. Asking for a consolidated References section at the end standardises attribution and makes the report look genuinely publication-ready.

Prompt Iteration Example

Original Step 3 system prompt (version 1):

Summarise the following search results. For each sub-question, write 2–3 sentences capturing the main points. Return plain text.

Problem:

The output was indistinguishable from asking the LLM to write about the topic from memory. Source URLs were discarded. The synthesis step (Step 4) received paraphrased training data dressed up as retrieved evidence.

Fix:

Switched to the JSON schema with `sub_question`, `fact`, and `source_url` fields. This forced the LLM to attribute every fact to a specific query and a specific URL, making it impossible to silently blend retrieved and recalled knowledge.

Result:

Step 4's synthesis became directly traceable to the search results, and the final report's References section became populated with real URLs rather than being empty.